NemoClaw: стек, безопасность и применение в закрытом контуре

1. Резюме для руководства

NemoClaw — это открытый стек NVIDIA (Apache 2.0), который оборачивает OpenClaw и запускает его внутри изолированного рантайма OpenShell, добавляя уровни безопасности, приватности и управляемости для постоянно работающих AI-агентов.[1][2][3][4]
Стек ориентирован на корпоративные сценарии: от on-prem до RTX-станций и DGX, при этом сам NemoClaw распространяется бесплатно, а платными являются только инфраструктура и, при желании, enterprise-поддержка NVIDIA.[2][5][1]

Ключевые преимущества для закрытого контура:

- Изоляция OpenClaw внутри OpenShell с политиками доступа к файловой системе, сети и процессам (Landlock, egress-контроль, аудит).[3][4][2]
- Поддержка OpenAI-совместимых LLM-провайдеров (Ollama, vLLM, NeMo/Nemotron), что критично для работы в изолированной среде через локальный inference.[4][6][2]
- Единый установочный поток (`curl | bash` + `nemoclaw onboard`), который поднимает sandbox, OpenClaw и конфигурирует провайдер моделей по декларативному blueprint'у.[2][4]

Для вашей задачи (лёгковесный Claw-подобный агент, локальный запуск, безопасная обвязка, поддержка OpenCode/Ollama/OpenAI-совместимого inference) NemoClaw + OpenClaw является сильным кандидатом, но в сочетании с дополнительным «тонким» локальным слоем (мини-обвязка

вокруг OpenCode и Ollama) можно построить ещё более лёгкий и аудируемый вариант.

2. Лицензирование и стоимость NemoClaw

2.1. Лицензия

Публичная документация и независимые обзоры указывают, что NemoClaw распространяется под лицензией Apache 2.0.[7][1][2] DigitalOcean-образ NemoClaw также фиксирует лицензии OpenShell и самого NemoClaw как Apache License 2.0.[7]

2.2. Стоимость

- Ядро NemoClaw (плагин, скрипты, blueprints) доступно бесплатно и с открытым исходным кодом.[5][1][2]
- NVIDIA отдельно предлагает платный enterprise-tier с управляемой инфраструктурой, комплаенсом и SLA-поддержкой; он не обязателен для self-hosted сценария.[1][5]
- Основные затраты в on-prem/закрытом контуре — это железо (GPU/CPU-кластеры), DEVOPS/MLOps и сопутствующий мониторинг, а не лицензия NemoClaw.[1]

3. Архитектура NemoClaw

3.1. Основные компоненты

Архитектура NemoClaw складывается из следующих слоёв:[2][3][4]

- **OpenClaw** — агентный движок (opchestrator, память, инструменты, интеграции).
- **OpenShell** — безопасный рантайм для агентов: создаёт sandbox (Docker/контейнеры), применяет политики, перехватывает системные вызовы и контролирует egress-трафик.[3][4]
- **NemoClaw Plugin/CLI** — тонкий слой, который устанавливается на хост и, через blueprint, поднимает

OpenShell-sandbox с OpenClaw, конфигурирует inference и политики.[8][4]
- **Inference Provider** — LLM-бэкенд: NeMo/Nemotron через NIM или vLLM, Ollama, внешний OpenAI-совместимый сервис, либо их комбинации.[6][4][2]

3.2. Поток установки и онбординга

NemoClaw использует двухэтапную установку:[4][2]

1. **Установка плагина и зависимостей**

   ```
   curl -fsSL https://nvidia.com/nemoclaw.sh | bash
   ```

   Скрипт устанавливает NemoClaw CLI, при необходимости доустанавливает Node.js, создаёт базовую структуру каталогов.
2. **Онбординг и создание sandbox'а**

   ```
   nemoclaw onboard
   ```

   Во время онбординга:
   - определяется blueprint (версионированный декларативный манифест);
   - OpenShell CLI создаёт Docker-sandbox и применяет политики безопасности;
   - внутрь sandbox'а разворачивается OpenClaw;
   - конфигурируется inference-провайдер (в т.ч. OpenAI-совместимый endpoint на Ollama/vLLM).[4]

Такой дизайн выносит тяжёлую бизнес-логику в blueprint, а сам NemoClaw CLI остаётся тонким и управляемым, что упрощает обновления и аудит.[4]

3.3. Работа в рантайме

После онбординга для управления окружением используются команды вида:[4]

```
nemoclaw my-assistant connect
nemoclaw my-assistant status
nemoclaw my-assistant logs --follow
```

OpenClaw работает внутри OpenShell-sandbox'а, генерируя запросы к LLM через сконфигурированный провайдер (например, Ollama или NeMo NIM); OpenShell на уровне ядра и сети контролирует, какие действия и куда может выполнять агент.[2][3][4]

4. Механизмы безопасности и управления

4.1. OpenShell как security-ядро

OpenShell выполняет роль «браузерной песочницы» для агентов:[3][4]

- **Перехват системных вызовов** — фильтрует доступ к файловой системе (Landlock/LSM), запрещая операции вне разрешённых директорий.
- **Контроль сетевого egress** — политики разрешённых доменов/адресов (allowlist) для HTTP-запросов агентов, с блокировкой всего остального.
- **Изоляция процессов** — OpenClaw и вспомогательные процессы запускаются внутри контейнера/неймспейса, изолированного от хоста.[3][4]

Это закрывает класс рисков «run arbitrary code на хосте», характерный для классического OpenClaw, который запускает всё внутри одного Node.js-процесса с доступом к хостовой системе.[9][10][3]

4.2. Политики и blueprints

Blueprint NemoClaw описывает:[4]

- ресурсы sandbox'а (объём CPU/RAM, лимиты);

- политики ФС (какие директории монтируются, в каком режиме: read-only/read-write);
- сетевые политики (разрешённые внешние сервисы);
- настройки inference-провайдера (тип, endpoint, ключи/токены);
- версионирование и контроль целостности (подпись blueprint'а, проверки при онбординге).[4]

За счёт этого можно создавать профили для разных ассистентов (HR-бот, DevOps-бот, внутренний Legal-бот) с разными наборами прав и LLM-моделей.

4.3. Аутентификация, авторизация и аудит

Над OpenShell и OpenClaw NemoClaw добавляет enterprise-механику:[1][2][3]

- интеграция с существующими SSO/IdP (OIDC/SAML) для аутентификации пользователей;
- ролевые модели доступа (RBAC) к ассистентам и их возможностям;
- аудит логов действий агентов (команды, сетевые обращения, попытки нарушений политик);
- централизованное управление конфигурацией blueprints и обновлениями окружения.

5. Возможности и интеграции

5.1. Модели и inference

NemoClaw через OpenClaw поддерживает:[6][2][4]

- **NeMo/Nemotron** в виде NIM-микросервисов (NVIDIA Inference Microservices);
- **Ollama** как локальный OpenAI-совместимый endpoint (/v1/chat/completions), включая модели вроде MiniMax M2.7:cloud, glm, Qwen и др.;[11][12][13]
- **vLLM / иные OpenAI-совместимые inference-сервера**, доступные в закрытом контуре;
- комбинации «модель по умолчанию + специализированные модели для code-/tool-интенсивных сценариев».

5.2. Интеграции и сценарии

OpenClaw даёт NemoClaw готовую экосистему интеграций:[9][2][3]

- GitHub/GitLab, Jira, Confluence, Slack/Teams, email и др.;
- операции с локальной ФС (внутри sandbox'а): чтение/запись файлов, запуск CLI-утилит;
- web-scraping, API-интеграции с внутренними системами (через HTTP-клиент под контролем OpenShell).

NemoClaw не ломает эти интеграции, а заворачивает их в контролируемую среду, где каждая команда и HTTP-запрос проходят через политики безопасности.[3][4]

6. Необходимые компоненты для закрытого контура

Для внедрения NemoClaw в изолированный контур (on-prem, без прямого доступа в интернет) необходимо заранее занести и сертифицировать:[7][2][4]

1. **Исходники/бинарии NemoClaw CLI и OpenShell**
   - Репозиторий NemoClaw (Apache 2.0).
   - Исходники/образы OpenShell (также Apache 2.0).[7]
2. **Образы контейнеров**
   - Базовый образ для OpenShell-sandbox'а.
   - Образ OpenClaw (Node.js-стек).
   - Образ/бандл для inference-сервера (Ollama или vLLM + модели).
3. **Инференс-стек**
   - **Ollama** или аналог, настроенный на работу только внутри контура (без обращения в публичный Ollama Cloud).
   - Локальные модели (например, MiniMax M-серии, Qwen, Llama, Nemotron), в виде весов/gguf/llama-формата, разрешённых к использованию.
4. **Инструменты интеграции**
   - Коннекторы к внутренним системам (Git, Jira, корпоративные API);

    - Внутренний PKI/секрет-менеджер для хранения ключей к этим системам.
5. **Мониторинг и логирование**
    - Адаптеры для существующего стека observability (Prometheus, Loki, ELK, др.) для логов NemoClaw/OpenShell/OpenClaw.

7. Альтернативы и гибридные решения

7.1. Альтернативы NemoClaw

1. **OpenClaw без NemoClaw**
    - Прямой запуск OpenClaw на серверах/рабочих станциях.
    - Плюсы: максимальная гибкость, огромная экосистема модулей.
    - Минусы: сложная кодовая база (~0,5M LoC), отсутствие системного sandbox'а, более высокий риск для ИБ.[10][9]
2. **NanoClaw**
    - Лёгковесный, контейнер-изолированный ассистент на Anthropic Agent SDK, MIT-лицензия.[14][10]
    - Плюсы: минимальный код (~500 строк ядра), жёсткая контейнерная изоляция, простая установка.
    - Минусы: жёсткая привязка к Anthropic (Claude) как основному LLM; локальные модели (Ollama) возможны только как инструменты, а не основной движок.[15][10][9]
3. **Самодельная связка OpenCode + Ollama/vLLM**
    - Использование OpenCode (или другого CLI-фреймворка) как основного интерфейса и планировщика, с прямым вызовом OpenAI-совместимого inference;
    - Плюсы: полный контроль, минимальный внешний код.
    - Минусы: нужно самостоятельно реализовывать память, tool-calling, безопасность, интеграции.

7.2. Выводы по выбору

- NemoClaw даёт максимально готовый enterprise-контур поверх OpenClaw, но тянет за собой сложность и объём стека.
- NanoClaw идеально по лёгкости и изоляции, но архитектурно завязан на Anthropic и не может «пересесть» на MiniMax/vLLM как основной LLM без серьёзного рефакторинга.[10][15][9]
- Самодельная обвязка OpenCode + Ollama/vLLM даёт максимальный контроль для закрытого контура, но потребует внутренней разработки.

Оптимальным выглядит **гибрид**: использовать идеи NemoClaw (sandbox + политики) и простоту NanoClaw/mini-Claw, построив собственный тонкий агентный хост, работающий поверх OpenCode и OpenAI-совместимого inference (Ollama/vLLM), с изоляцией на уровне контейнеров или OpenShell.

8. Требования и архитектура предлагаемого «лёгкого Claw»

8.1. Функциональные требования

- Локальный запуск на Linux/Windows/macOS.
- OpenAI-совместимый LLM-backend: Ollama, vLLM, NeMo-NIM, локальный OpenAI-совместимый шлюз.
- Интеграция с OpenCode/CLI для разработки и управления агентами.
- ФС/процесс/сеть-изоляция (через Docker/OpenShell/Podman).
- Поддержка нескольких агентов/профилей с разными правами.

8.2. Предлагаемая архитектура

1. **Agent Host (LightClaw)**
    - Небольшой сервис на TypeScript/Go/Python (по выбору команды), который:
        - принимает запросы от пользователей (CLI, HTTP, мессенджеры);

        - хранит состояние сессий/память (SQLite/PostgreSQL);
        - организует вызовы LLM через OpenAI-совместимый API;
        - оркестрирует инструменты/CLI-команды.
2. **Sandbox Layer**
    - Вариант A: Docker/Podman + минимальный runtime (как у NanoClaw).
    - Вариант B: интеграция с OpenShell/Open-source аналогом для политик на уровне ядра (по образцу NemoClaw).[3][4]
3. **Inference Layer**
    - Ollama с моделями (MiniMax M2.7, другие) в режиме offline/air-gapped.
    - vLLM или кастомный server, если нужны другие форматы/модели.
4. **Developer Experience**
    - OpenCode как основной инструмент работы разработчиков с репозиторием LightClaw: генерация skill'ов, тестирование, CI/CD.
    - CLI-обвязка по образцу `nemoclaw onboard`, но в минимальном варианте.

9. План разработки гибридной системы

9.1. Этап 0 — Discovery и протоколы

- Уточнение требований ИБ (список запрещённых действий, необходимый аудит).
- Выбор основного языка реализации (Python/Go/TypeScript).
- Утверждение протокола взаимодействия с LLM (OpenAI-совместимый `/v1/chat/completions`).

9.2. Этап 1 — PoC на одной машине

1. Развернуть Ollama и несколько локальных моделей (включая MiniMax M-серии при наличии локального варианта).
2. Реализовать минимальный Agent Host:
    - HTTP-API `POST /chat` → проксирование на Ollama;

    - простая память (SQLite);
    - текстовый лог запросов/ответов.
3. Обернуть Agent Host в Docker-контейнер, ограничив доступ к ФС и сети (вручную через Docker-параметры).
4. Подключить OpenCode для генерации и правки skill-модулей (набор CLI-инструментов, которые агент может вызывать).

9.3. Этап 2 — Политики безопасности и multi-agent

1. Формализовать политики безопасности (YAML/JSON):
    - разрешённые команды;
    - разрешённые директории;
    - разрешённые внешние хосты.
2. Встроить движок политик в Agent Host (или интегрировать OpenShell при наличии):
    - перехват и проверка всех попыток запустить CLI/доступиться к ФС/сети;
    - логирование и алертинг нарушений.
3. Добавить поддержку нескольких агентов/профилей с разными наборами политик и моделей.

9.4. Этап 3 — Интеграции и эксплуатация в контуре

1. Реализовать коннекторы к внутренним системам (Git, Jira, CI, внутренние REST-API).
2. Интегрировать с корпоративным SSO и секрет-менеджером.
3. Включить метрики и логирование в существующий мониторинг (Prometheus/ELK).
4. Провести пилот с ограниченным числом пользователей и чётко очерченным набором задач (например, DevOps-ассистент).
5. После пилота — hardening, review ИБ и выведение в промышленную эксплуатацию.

10. Ограничения и замечания

- NemoClaw находится в alpha/early-stage, что требует осторожного подхода к обновлениям и возможной нестабильности; для закрытого периметра

предпочтительно зафиксировать проверенную версию и обновлять только после тестов.[8][4]
- NanoClaw теоретически можно форкнуть и заменить Anthropic Agent SDK на OpenAI-совместимый backend, но это уже не будет «поддерживаемый NanoClaw», а внутренний форк, который нужно сопровождать самостоятельно.[15][9][10]
- Любая интеграция с облачными моделями (Ollama:cloud, OpenAI, Anthropic) в закрытом контуре должна проходить отдельное согласование с ИБ; для большинства сценариев разумно ориентироваться на полностью локальный inference.

References

1. NemoClaw: What It Is, How It Works, and Alternatives (2026 Guide) - NemoClaw is NVIDIA's open-source AI agent platform for enterprises, unveiled at GTC 2026. Learn what...
2. NemoClaw - NemoClaw is an open source stack that simplifies running OpenClaw always-on assistants—with a single...
3. NemoClaw: NVIDIA's Open Source Stack for Running AI ... - NemoClaw is an open source stack from NVIDIA that wraps OpenClaw (the popular always-on AI assistant...
4. Getting Started With NemoClaw: Install, Onboard, and ... - Learn how to install and onboard with NemoClaw. Avoid common mistakes in this OpenClaw tutorial on N...
5. NemoClaw Just Replaced $2K Enterprise Tools — For FREE! - NVIDIA just dropped NemoClaw — and it's about to destroy every $2,000/month enterprise AI tool on th...
6. NemoClaw | DGX Spark - NVIDIA NemoClaw is an OpenClaw plugin that packages OpenShell with an AI agent: it includes the nemo...
7. NemoClaw (Alpha) | DigitalOcean Documentation - NemoClaw is an open-source framework for building and deploying intelligent AI agents capable of int...
8. GitHub - NVIDIA/NemoClaw: NVIDIA plugin for secure... - NVIDIA NemoClaw is an open source alpha-stage plugin that simplifies running OpenClaw always-on AI a...

9. GitHub All-Stars #14: NanoClaw - We're looking at NanoClaw: a lightweight, container-isolated personal AI assistant that connects to ...
10. GitHub All-Stars #14: NanoClaw - Written in TypeScript, MIT-licensed, and built around one radical premise: the software that hosts p...
11. minimax-m2.7 - MiniMax M2.7 is the first model in the M2-series to deeply participate in its own evolution, capable...
12. ollama launch claude --model minimax-m2.7:cloud Use it ... - MiniMax-M2.7 is now available on Ollama's cloud. made for coding and agentic tasks Try it inside ...
13. Using OpenClaw with Ollama: Building a Local Data Analyst - Learn how to build a fully local AI data analyst using OpenClaw and Ollama that orchestrates multi-s...
14. NanoClaw solves one of OpenClaw's biggest security issues - Enter NanoClaw, a lighter, more secure version which debuted under an open source MIT License on Jan...
15. Comparing NanoClaw and OpenClaw, two major open-source AI ... - NanoClaw or OpenClaw? This is the choice every developer looking to build a personal AI assistant wi...